

삼성청년 SW·AI아카데미

B형 특강

<알림>

본 강의는 삼성청년SW·AI아카데미의 콘텐츠로
보안서약서에 의거하여
강의 내용을 어떠한 사유로도 임의로 복사, 촬영,
녹음, 복제, 보관, 전송하거나
허가 받지 않은 저장매체를
이용한 보관, 제3자에게 누설, 공개,
또는 사용하는 등의 행위를 금합니다.

Pro 연습 문제. 퍼즐 맞추기

퍼즐 맞추기 - 문제 이해

퍼즐 맞추기는 게임판에 규칙에 맞는 위치에 퍼즐 조각들을 올려놓는 게임이다.

세로 길이가 mRows이고 가로 길이가 mCols인 직사각형 모양의 퍼즐판이 있다.

이 게임 판은 1*1 크기의 셀들 mRows * mCols개로 구성된다.

각 셀에는 1~5 중 하나의 숫자가 적혀있다.

[Fig. 1]은 mRows가 10이고, mCols가 6일 때 10 * 6 크기의 게임 판을 보여준다.

2	2	3	3	2	3
2	3	2	4	3	2
1	2	3	3	2	2
2	3	4	4	5	5
1	2	3	3	3	4
1	2	3	3	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

[Fig. 1]

퍼즐판 내의 셀의 위치는 (행 번호, 열 번호)로 표현한다.

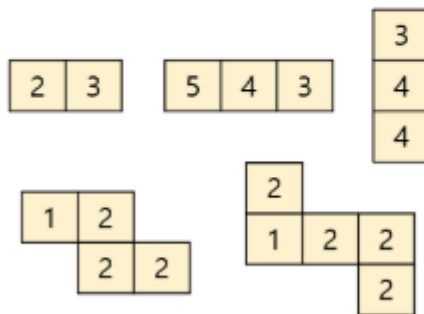
위치(0, 4)에는 숫자 2가 적혀있고, 위치(9, 5)에는 숫자 3이 적혀있다.

[퍼즐 조각]

하나의 조각은 1*1 크기의 정사각형 블록들이 여러 개 연결되어 모양을 이루고 있다.

주어지는 조각의 모양은 [Fig. 2]와 같이 다섯 종류가 있다.

각 블록에는 1~5 중 하나의 숫자가 적혀있다.



[Fig. 2]

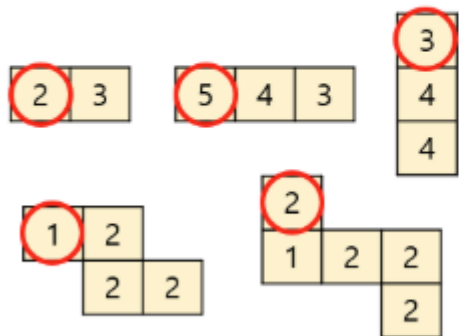
퍼즐 맞추기 - 문제 이해

[퍼즐 조각 놓기]

플레이어는 주어진 조각을 게임판에 올려놓아야 한다.

조각을 올려 놓은 위치는 조각의 가장 윗부분 중 가장 왼쪽의 블록을 기준으로 퍼즐판의 위치로 표현된다.

모양 별로 기준이 되는 블록은 [Fig. 3] 에서 빨간 원으로 보여준다.



[Fig. 3]

만약 [Fig. 4]와 같이 조각이 퍼즐판에 놓여있는 경우, 조각을 위치 (3, 1)에 놓았다고 말한다.

2	2	3	3	2	3
2	3	2	4	3	2
1	2	3	3	2	2
2	2	4	4	5	5
1	1	2	2	3	4
1	2	3	2	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

[Fig. 4]

퍼즐 맞추기 - 문제 이해

[규칙]

플레이어는 주어진 조각을 아래 규칙에 맞는 위치에만 올려놓을 수 있다.

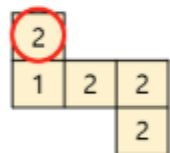
[규칙 1]

주어진 조각 내의 각각의 블록들에 대하여 블록의 숫자와 그 블록이 맞닿은 퍼즐판 셀의 숫자의 차이(조각의 블록의 숫자 - 게임판 셀의 숫자)가 모두 동일한 경우에만 올려 놓을 수 있다.

아래 [Fig.5]의 조각이 주어졌을 때 위치 (2, 1)에 놓게 되면 퍼즐판의 맞닿는 셀들의 숫자들은 (2, 3, 4, 4, 3)이므로, 각각 블록의 차들은 (0, -2, -2, -2, -1)이 된다.

하지만 위치 (3, 1)에 놓게 되면 각각 블록의 차들은 모두 -1로 동일하다.

따라서 해당 조각은 위치 (2, 1)에는 놓을 수 없지만 위치 (3, 1)에는 놓을 수 있다.



2	2	3	3	2	3
2	3	2	4	3	2
1	2	3	3	2	2
2	3	4	4	5	5
1	2	3	3	3	4
1	2	3	3	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

2-2 = 0
1-3 = -2
2-4 = -2
2-4 = -2
2-3 = -1

2	2	3	3	2	3
2	3	2	4	3	2
1	2	3	3	2	2
2	3	4	4	5	5
1	2	3	3	3	4
1	2	3	3	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

2-3 = -1
1-2 = -1
2-3 = -1
2-3 = -1
2-3 = -1

퍼즐 맞추기 - 문제 이해

[규칙 2]

퍼즐판에 주어진 조각을 놓을 때, 그 조각의 일부가 이미 놓여있는 조각의 일부와 중첩되게 놓을 수 없다.

[Fig. 6]와 같이 퍼즐에 조각이 놓여 있는 상황에서 주어진 조각을 위치 (4, 1)에 놓을 경우 일부가 겹치게 된다.
위치 (6,0)과 같이 겹치는 부분이 없는 곳에만 놓을 수 있다.

2	2	3	3	2	3
2	3	2	4	3	2
1	2	3	3	2	2
2	2	4	4	5	5
1	1	2	2	3	4
1	2	2	2	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

2	2	3	3	2	3
2	3	2	4	3	2
1	2	3	3	2	2
2	2	4	4	5	5
1	1	2	2	3	4
1	2	3	2	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

[Fig. 6]

퍼즐 맞추기 - 문제 이해

[규칙 3]

주어진 조각을 놓을 수 있는 위치가 둘 이상인 경우, 우선순위가 가장 높은 위치에 놓아야 한다.

우선순위는 놓을 수 있는 위치의 **행의 번호가 작을수록, 행의 번호가 같다면 열의 번호가 작을수록 높다.**

[Fig. 7]와 같이 조각이 주어지는 경우, 퍼즐판에 놓을 수 있는 위치의 개수는 빨간 박스로 보여주듯이 총 6개이다.

만약 두번째 이미지와 같이 녹색 조각들이 놓여 있는 경우 놓을 수 있는 위치들 중 우선순위에 따라 위치 (2, 1)에 주어진 조각을 놓아야 한다.

만약 세번째 이미지와 같이 녹색 조각들이 놓여 있는 경우에는 우선순위에 따라 위치 (3, 3)에 주어진 조각을 놓아야 한다.

4	5	5
---	---	---

2	2	3	3	2	3
2	3	2	4	3	2
1	2	3	3	2	2
2	3	4	4	5	5
1	2	3	3	3	4
1	2	3	3	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

2	2	3			3
2	3	2	4		
1	2	3	3	2	2
2	3	4	4	5	5
1	2	3	3	3	4
1	2	3	3	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

2		3	3	2	3
2				3	2
1	2	3		2	2
2		4	4	5	5
1				3	4
1	2	3		3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

[Fig. 7]

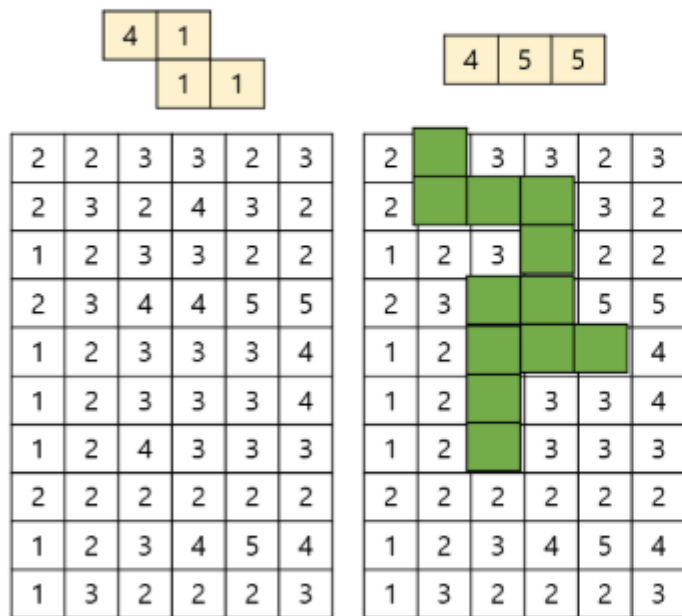
퍼즐 맞추기 - 문제 이해

[규칙 4]

조각이 주어졌을 때 놓을 위치가 존재하지 않는 경우 그 조각은 사라진다.

[Fig. 8]와 같이 주어진 조각이 1번 규칙을 만족하는 위치가 없거나,

위치 존재하더라도 이미 다른 조각들이 놓여있어서 더 이상 놓을 곳이 없다면 그대로 사라진다.



[Fig. 8]

플레이어가 주어지는 조각을 위 규칙에 맞게 위치 시킬 수 있도록 도움을 줄 수 있는 프로그램을 작성하라.

퍼즐 맞추기 - API

```
public void init(int mRows, int mCols, int[][] mCells)
```

각 테스트 케이스의 처음에 호출된다.

- 크기가 $mRows * mCols$ 인 퍼즐판이 주어진다.
- 퍼즐판의 행의 개수는 $mRows$ 이고, 열의 개수는 $mCols$ 가 된다.
- 퍼즐판 내에서 i 번째 행, j 번째 열에 위치한 셀에 적힌 숫자는 $mCells[i][j]$ 로 주어진다.
- 초기에는 퍼즐판에 어떠한 조각도 놓여있지 않다.

Parameters

- $mRows$: 퍼즐판의 행의 개수 ($5 \leq mRows \leq 40$)
- $mCols$: 퍼즐판의 열의 개수 ($5 \leq mCols \leq 30$)
- $mCells[i][j]$: 퍼즐판 내의 셀에 적힌 숫자 ($1 \leq mCells[i][j] \leq 5$)

퍼즐 맞추기 - API

```
public Main.Result putPuzzle(int[][] mPuzzle)
```

조각의 정보가 주어질 때 위 규칙에 맞는 위치를 찾아 놓고 그 위치를 반환해야 한다.

- 만약, 놓을 위치가 없는 경우, Result.row와 Result.col 값을 -1로 설정한 후 반환한다.
- 조각의 모양과 각 블록에 적혀있는 숫자의 정보는 mPuzzle로 주어진다.
- (조각의 모양은 위에서 언급한 5가지 중 한가지로 주어진다.)

mPuzzle[][]의 값은 블록에 적혀있는 숫자이고, 값이 0인 경우 해당 위치는 블록이 없음을 의미한다.

- **주어진 조각의 가장 윗부분 중 가장 왼쪽의 블록의 숫자는 mPuzzle[0][0]에 위치함을 보장한다.**
- 다음은 주어진 조각에 대해서 mPuzzle 값으로 주어지는 예이다.

mPuzzle

[0][0]	[0][1]	[0][2]	2	3		2	0	0
[1][0]	[1][1]	[1][2]				1	2	2
[2][0]	[2][1]	[2][2]				0	0	2

Parameters

- mPuzzle : 주어지는 조각의 정보 ($0 \leq \text{mPuzzle}[][] \leq 5$)

Returns

- Result.row : 주어진 조각이 규칙에 맞게 놓이는 위치의 행 번호(놓을 위치가 없는 경우 -1)
- Result.col : 주어진 조각이 규칙에 맞게 놓이는 위치의 열 번호(놓을 위치가 없는 경우 -1)

퍼즐 맞추기 - API

```
public void clearPuzzles()
```

게임판 내의 조각들을 모두 없애 게임판을 초기 상태로 만든다.

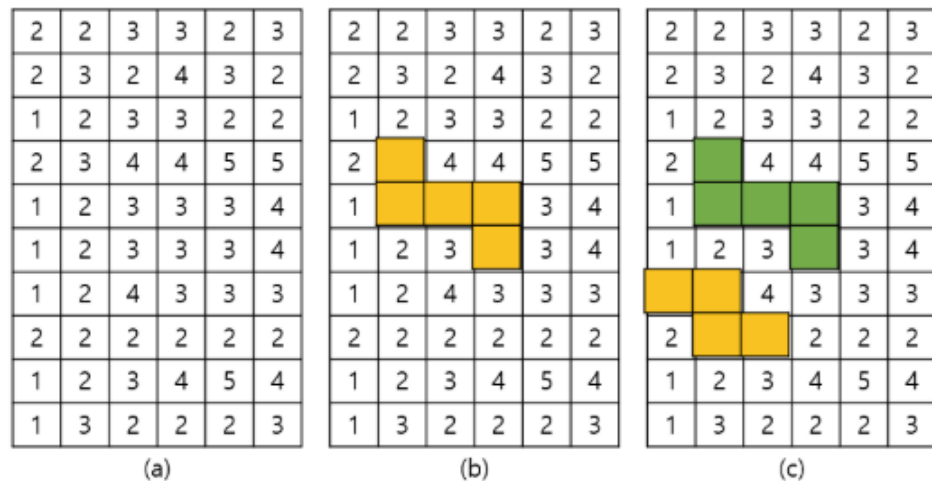
[제약사항]

1. 각 테스트 케이스 시작 시 init() 함수가 호출된다.
2. 각 테스트 케이스에서 putPuzzle() 함수는 최대 100,000 회 호출된다.
3. 각 테스트 케이스에서 clearPuzzles() 함수는 최대 1,000 회 호출된다.
4. 각 테스트 케이스에서 주어지는 조각이 1번 규칙에 만족하는 위치는 최대 500개, 평균 70개 이하이다.

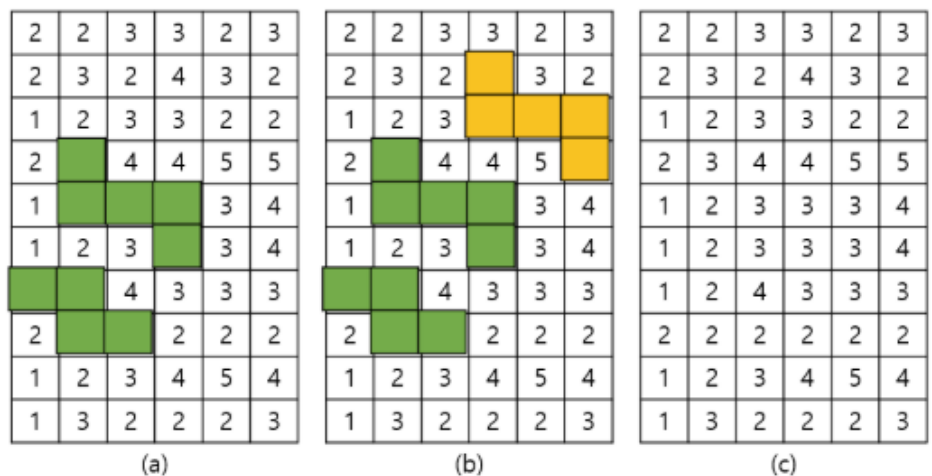
퍼즐 맞추기 - 예시

[예제]

Order	Function	Return	Figure
1	init(10, 6, { ... })		Fig.8 (a)
2	putPuzzle({ [2,0,0], [1,2,2], [0,0,2] })	(Result.row, Result.col) (3, 1)	Fig.8 (b)
3	putPuzzle({ [1,2,0], [0,2,2], [0,0,0] })	(6, 0)	Fig.8 (c)
4	putPuzzle({ [1,2,0], [0,2,2], [0,0,0] })	(-1, -1)	Fig.9 (a)
5	putPuzzle({ [3,0,0], [2,1,1], [0,0,4] })	(1, 3)	Fig.9 (b)
6	clearPuzzles()		Fig.9 (c)
7	putPuzzle({ [3,0,0], [2,0,0], [5,0,0] })	(1, 4)	Fig.10 (a)
8	putPuzzle({ [2,0,0], [2,3,4], [0,0,5] })	(4, 0)	Fig.10 (b)
9	putPuzzle({ [4,0,0], [4,0,0], [4,0,0] })	(4, 3)	Fig.10 (c)
10	putPuzzle({ [2,2,2], [0,0,0], [0,0,0] })	(7, 0)	Fig.11 (a)
11	putPuzzle({ [3,3,3], [0,0,0], [0,0,0] })	(7, 3)	Fig.11 (b)
12	putPuzzle({ [1,3,0], [0,0,0], [0,0,0] })	(1, 2)	Fig.11 (c)
13	putPuzzle({ [4,1,0], [0,1,1], [0,0,0] })	(-1, -1)	Fig.12 (a)
14	putPuzzle({ [2,4,0], [0,0,0], [0,0,0] })	(9, 0)	Fig.12 (b)
15	clearPuzzles()		Fig.12 (c)



[Fig. 8]



[Fig. 9]

퍼즐 맞추기 - 예시

2	2	3	3	2	3
2	3	2	4		2
1	2	3	3		2
2	3	4	4		5
1	2	3	3	3	4
1	2	3	3	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

(a)

2	2	3	3	2	3
2	3	2	4		2
1	2	3	3		2
2	3	4	4		5
	2	3	3	3	4
				3	4
1	2		3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

(b)

2	2	3	3	2	3
2	3	2	4		2
1	2	3	3		2
2	3	4	4		5
	2	3		3	4
				3	4
1	2		3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

(c)

[Fig. 10]

2	2	3	3	2	3
2	3				2
1	2	3	3		2
2	3	4	4		5
	2	3		3	4
				3	4
1	2			3	3
1	2	3	4	5	4
1	3	2	2	2	3

(a)

2	2	3	3	2	3
2	3				2
1	2	3	3		2
2	3	4	4		5
	2	3		3	4
				3	4
1	2			3	3
1	2	3	4	5	4
1	3	2	2	2	3

(b)

2	2	3	3	2	3
2	3	2	4	3	2
1	2	3	3	2	2
2	3	4	4	5	5
1	2	3	3	3	4
1	2	3	3	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

(c)

[Fig. 12]

2	2	3	3	2	3
2	3	2	4		2
1	2	3	3		2
2	3	4	4		5
	2	3		3	4
				3	4
1	2			3	3
1	2	3	4	5	4
1	3	2	2	2	3

(a)

2	2	3	3	2	3
2	3	2	4		2
1	2	3	3		2
2	3	4	4		5
	2	3		3	4
				3	4
1	2			3	3
1	2	3	4	5	4
1	3	2	2	2	3

(b)

2	2	3	3	2	3
2	3				2
1	2	3	3		2
2	3	4	4		5
	2	3		3	4
				3	4
1	2			3	3
1	2	3	4	5	4
1	3	2	2	2	3

(c)

[Fig. 11]

설계하기

삼성청년SW·AI아카데미

putPuzzle() 계산

```
for(int i = 0; i < mRows; i++) {  
    for(int j = 0; j < mCols; j++) {  
        for(블록의 방향/개수) {  
            if(여기 둘 수 있다면) {  
                여기 둔다()  
                return i, j  
            }  
        }  
    }  
}
```

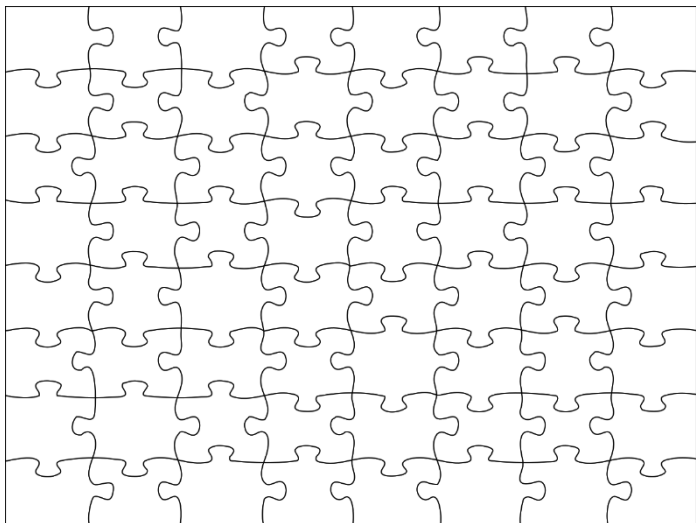
- mRows : 최대 40
- mCols : 최대 30
- 블록의 방향 : 최대 5
- 둘 수 있는지 확인 : 최대 5
- 둔다 기록 : 최대 5 → 계산에서 제외
- 호출 횟수 : 최대 100,000회

❖ 호출 횟수 : 100,000

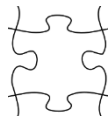
- $100,000 \times (40 \times 30 \times 5) = 600,000,000$ (TLE)

❖ 퍼즐판의 정보는 변하지 않는다.

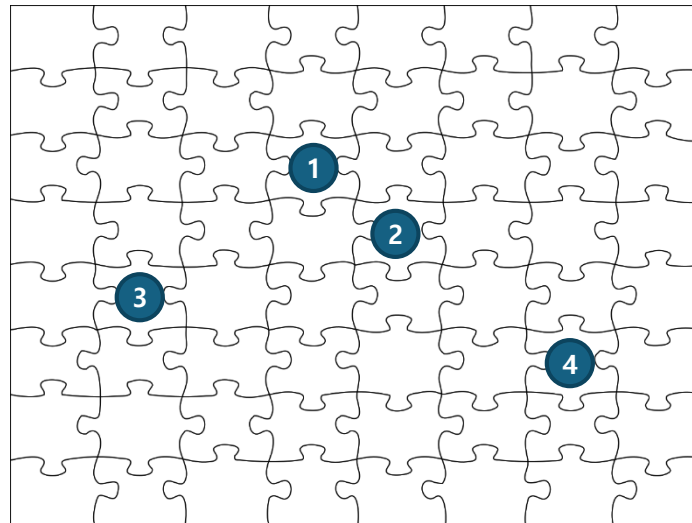
- 특정 퍼즐을 놓을 수 있는 위치는 항상 고정되어 있다.
- 아래 예시를 확인해보며 생각해보자.



이런 모양의 퍼즐 판에서



이런 조각을 뒤야 한다면?



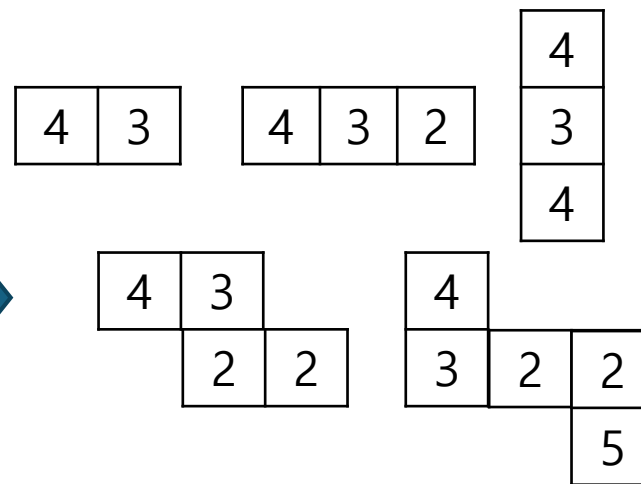
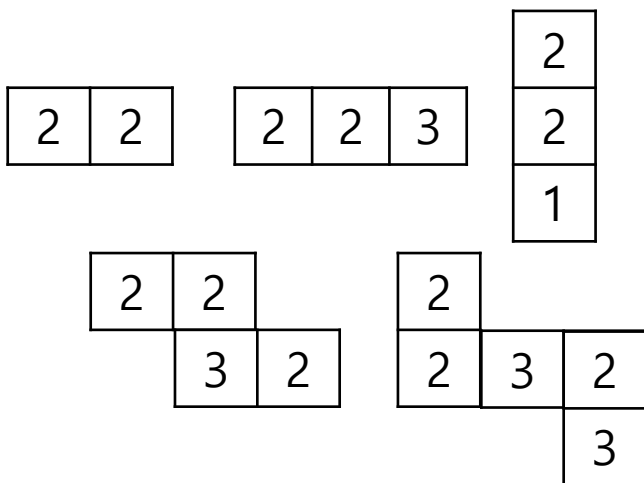
다 둘러보는게 아니라,
이 모양을 둘 수 있는 곳만 확인한다.

❖ 즉, 어떤 특정 조각을 어디에 둘 수 있는지를 미리 알고 있으면, 불필요한 탐색을 줄일 수 있다.

Hashing

❖ 퍼즐판의 정보를 토대로 둘 수 이쓴 모든 조각의 가능성을 기록해두자.

2	2	3	3	2	3
2	3	2	4	3	2
1	2	3	3	2	2
2	3	4	4	5	5
1	2	3	3	3	4
1	2	3	3	3	4
1	2	4	3	3	3
2	2	2	2	2	2
1	2	3	4	5	4
1	3	2	2	2	3

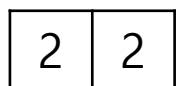


이런 블록들은 (0, 0) 위치에 둘 수 있다! 이런 블록들은 (1, 3) 위치에 둘 수 있다!

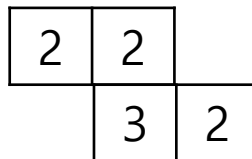


이걸 어떻게 하지?

- ❖ 일반적으로 저장할 수 없는 것 → 저장할 수 있는 것으로 바뀌야 한다
- ❖ **Hashing** 기법 사용 : 특정 패턴이나 종바에 고유 식별 번호를 부여하여 데이터를 관리



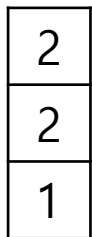
0번 모양의 22번



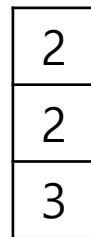
3번 모양의 2232번



1번 모양의 223번



2번 모양의 221번



2번 모양의 223번

- ❖ 조각의 상태에 따라 같은 식별 번호가 생성될 수 있으니, 모양(shape)별로 관리 필요

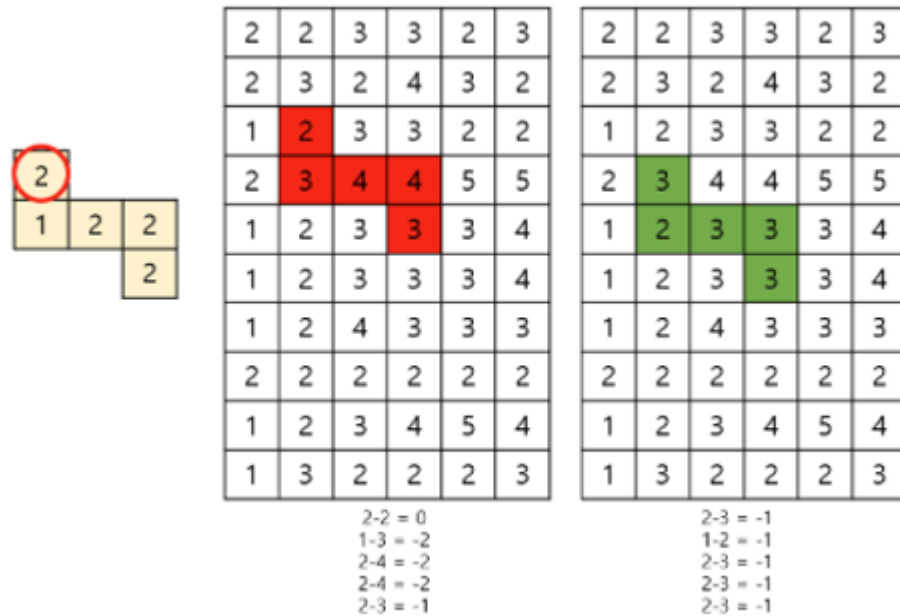
방향 그래프

- ❖ 방향 그래프이기 때문에 출발지 $\rightarrow$ 도착지 $\neq$ 도착지 $\rightarrow$ 출발지 이다.
- ❖ 즉, 만약 **"양방향"/"무향" 그래프**였다면, dijkstra(mHub) 호출 한번으로 해결될 수 있다.
- ❖ 이 로직으로 문제를 어떻게 바꾸면 / 적용하면 문제를 효율적으로 해결할 수 있을까?

규칙 #1

❖ 퍼즐판의 정보를 기록했는데, 입력되는 퍼즐 조각 모양과 일치하지 않을 수 있다.

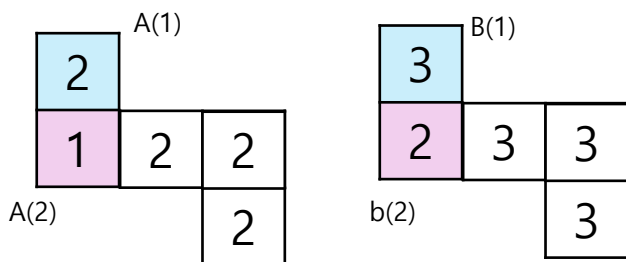
- 아래 예시에서는 4번 모양의 32333번이 (2, 1)위치에 둘 수 있다는 것을 알고 있지만, 입력되는 조각 모양은 4번의 21222번 모양이기 때문에, 일치하지 않는다.



- 21222번, 32333번, 43444번 모양을 둘 수 있는 곳들을 모두 확인하면 된다.
 - 다만, 이들이 "동일하다" 라는 것을 알고 있으니, **하나의 식별 번호로 관리**해주면 좋다.

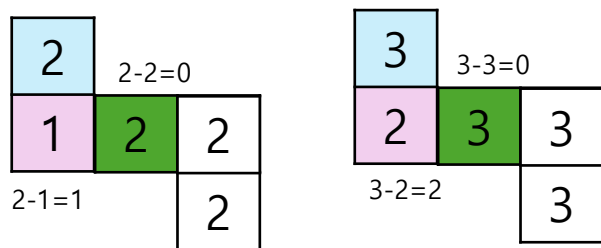
❖ 아래 두 개의 퍼즐은 같은 데에 끼워질 수 있는 조각이다.

- 각 위치들의 차이는 동일하다.
- $A(1) - B(1) == A(2) - B(2)$



❖ 방정식을 살짝 수정하면, 다음과 같다.

- $A(1) - A(2) == B(1) - B(2)$
- 즉, 조각 내의 서로 이웃된 블록의 차이는 동일하다.



Hashing (re)

❖ 이제 다른 값을 가진 퍼즐들도 모두 동일하게 인식되도록 하나의 번호로 hashing이 가능하다.

2	2
---	---

0번 모양의 0번

$$2-2 = 0$$

2	2	3
---	---	---

1번 모양의 -1번

$$2-2 = 0$$

$$2-3 = -1$$

2	2	
	3	2

3번 모양의 -10번

$$2-2 = 0$$

$$2-3 = -1$$

$$2-2 = 0$$

❖ init()

- 퍼즐판의 모든 정보를 모든 모양에 맞춰 hashing 후, 위치들을 저장한다.

```
for(int i = 0; i < mRows; i++) {  
    for(int j = 0; j < mCols; j++) {  
        for(shape의 개수(5)) {  
            hash = hashFunction(i, j, k, mCells)  
            이 shape의 이 hash번호의 블록은 (i,j)위치에 둘 수 있다!        }  
    }  
}
```

· 시간복잡도 :

- $O(mRows \times mCols \times 5 \times 5) = 30,000$

❖ putPuzzle()

- 주어진 퍼즐 조각의 모양 (shape)를 찾는다.
- 해당 정보를 가지고 이 조각의 hash를 찾는다.
- 이 모양, 이 hash번호를 가지는 조각을 둘 수 있는 곳을 모두 확인하며
 - 만약 둘 수 있다면 두고, ret의 값을 갱신, 탐색을 중단한다.
 - 만약 둘 수 없다면, 다음 가능성의 위치를 확인한다.

• 시간복잡도 :

- $100,000 \times O(\text{shape 모양의 hash 번호의 조각을 둘 수 있는 경우의 수})$

[제약사항]

1. 각 테스트 케이스 시작 시 init() 함수가 호출된다.
2. 각 테스트 케이스에서 putPuzzle() 함수는 최대 100,000 회 호출된다.
3. 각 테스트 케이스에서 clearPuzzles() 함수는 최대 1,000 회 호출된다.
4. 각 테스트 케이스에서 주어지는 조각이 1번 규칙에 만족하는 위치는 최대 500개, 평균 70개 이하이다.

- 최대 500개로 가정 = 50,000,000
- 평균 70개로 가정 = 7,000,000

❖ clearPuzzles()

- 퍼즐 조각들을 둔 기록을 초기화
- 시간복잡도 :
 - $1,000 \times O(mRows \times mCols) = 1,200,000$